UD4. Integridade e consistencia dos datos

A integridade e a consistencia son fundamentais para garantir que a información almacenada nunha base de datos sexa correcta, válida e coherente. Nesta parte da unidade estudaranse as restricións que permiten controlar os datos introducidos, os mecanismos para evitar operacións incorrectas e a combinación de diferentes técnicas SQL nun proceso completo de manipulación de información.

1. Integridade e consistencia da información

1.1. Concepto de integridade dos datos

A **integridade dos datos** é a propiedade que garante que a información almacenada nunha base de datos sexa correcta, válida e cumpra as regras establecidas no seu deseño.

Unha base de datos debe evitar situacións como:

- Introducir dous usuarios co mesmo identificador.
- Gardar un alumno sen nome ou sen os datos obrigatorios.
- Matricular un alumno nun curso que non existe.
- Asignar un prezo negativo a un produto.
- Introducir valores duplicados nun campo que debe ser único.
- Eliminar un rexistro que está sendo referenciado por outras táboas.

As restricións de integridade permiten que sexa o propio sistema xestor de bases de datos quen comprobe estas condicións, evitando depender exclusivamente das aplicacións que acceden á información.

1.2. Tipos de integridade

Podemos distinguir diferentes tipos de integridade:

Tipo	Descrición	Exemplo
Integridade de entidade	Identifica de maneira única cada rexistro dunha táboa.	Un alumno ten un identificador único.
Integridade referencial	Garante que as relacións entre táboas sexan válidas.	Unha matrícula referencia un alumno existente.
Integridade de dominio	Limita os valores que pode almacenar un campo.	A idade debe ser un número positivo.
Integridade definida polo negocio	Aplica regras específicas da organización.	Un pedido non pode ter un importe negativo.

2. Restricións de integridade en MariaDB

As restricións ou *constraints* son condicións que se definen sobre as columnas ou táboas para controlar os valores que se poden almacenar.

MariaDB comproba estas condicións cando se executan operacións como `INSERT`, `UPDATE` ou `DELETE`. Se unha operación incumpre unha restrición, o sistema devolve un erro e non permite aplicar o cambio incorrecto.

2.1. PRIMARY KEY

A restrición `PRIMARY KEY` identifica de forma única cada rexistro dunha táboa.

Unha clave primaria debe cumprir dúas condicións:

- Non pode conter valores duplicados.
- Non pode admitir valores `NULL`.

Cada táboa pode ter unha única clave primaria, aínda que esta pode estar formada por varias columnas.

```
CREATE TABLE alumnos (
    id INT AUTO_INCREMENT,
    dni VARCHAR(9) NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    apellidos VARCHAR(150),
    PRIMARY KEY (id)
) ENGINE=InnoDB;
```

Neste exemplo, o campo `id` identifica cada alumno de maneira única. O atributo `AUTO_INCREMENT` permite xerar automaticamente os identificadores.

Clave primaria composta

Unha clave primaria pode estar formada por varias columnas. Neste caso, a combinación dos seus valores debe ser única.

```
CREATE TABLE matriculas (
    alumno_id INT,
    asignatura_id INT,
    fecha DATE NOT NULL,
    PRIMARY KEY (alumno_id, asignatura_id)
) ENGINE=InnoDB;
```

A combinación de `alumno_id` e `asignatura_id` impide que un alumno se matricule dúas veces na mesma asignatura.

2.2. FOREIGN KEY

A restrición `FOREIGN KEY` establece unha relación entre dúas táboas e garante a integridade referencial.

Unha clave foránea referencia normalmente a clave primaria doutra táboa. O seu valor debe corresponder cun rexistro existente na táboa referenciada, salvo que se permita o valor `NULL`.

```
CREATE TABLE cursos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE alumnos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    curso_id INT,
    CONSTRAINT fk_alumnos_cursos
        FOREIGN KEY (curso_id)
        REFERENCES cursos(id)
) ENGINE=InnoDB;
```

Neste exemplo, cada alumno pode estar asociado a un curso. O valor de `curso_id` debe existir previamente na táboa `cursos`, ou ser `NULL` porque a columna non está definida como `NOT NULL`.

Se se intenta inserir un alumno cun identificador de curso inexistente, MariaDB devolverá un erro de integridade referencial.

```
-- Supoñamos que non existe o curso 99.

INSERT INTO alumnos (nombre, curso_id)
VALUES ('Ana López', 99);
```

> Para crear unha clave foránea, as táboas deben utilizar un motor compatible, como InnoDB, e as columnas relacionadas deben ter tipos compatibles. A columna referenciada debe estar indexada adecuadamente.

Políticas ON DELETE e ON UPDATE

MariaDB permite definir o comportamento que se aplicará cando se elimine ou modifique un rexistro referenciado por outras táboas.

Opción	Comportamento
RESTRICT	Impide eliminar ou modificar o rexistro referenciado cando existen dependencias.
NO ACTION	Equivale a RESTRICT en InnoDB.
CASCADE	Propaga a eliminación ou modificación aos rexistros relacionados.
SET NULL	Establece a NULL a columna da clave foránea, se o permite.

Exemplo de eliminación en cascada:

```
CREATE TABLE matriculas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    alumno_id INT NOT NULL,
    fecha DATE NOT NULL,
    FOREIGN KEY (alumno_id)
        REFERENCES alumnos(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Se se elimina un alumno, tamén se eliminarán as matrículas asociadas a el.

> **Precaución:** utilizar `ON DELETE CASCADE` pode provocar a eliminación de moitos rexistros relacionados. Debe empregarse só cando ese comportamento corresponda coas regras do sistema.

2.3. UNIQUE

A restrición `UNIQUE` garante que os valores dunha columna, ou dunha combinación de columnas, non se repitan.

A diferenza da clave primaria, unha táboa pode ter varias restricións `UNIQUE`.

```
CREATE TABLE usuarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    nome_usuario VARCHAR(50) NOT NULL UNIQUE
) ENGINE=InnoDB;
```

Neste exemplo, non poden existir dous usuarios co mesmo correo electrónico nin co mesmo nome de usuario.

Tamén se pode definir unha restrición única sobre varias columnas:

```
CREATE TABLE reservas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    sala_id INT NOT NULL,
    fecha DATE NOT NULL,
    hora TIME NOT NULL,
    UNIQUE (sala_id, fecha, hora)
) ENGINE=InnoDB;
```

A combinación de sala, data e hora non pode repetirse. Así evítase que se rexistren dúas reservas para a mesma sala no mesmo momento.

En MariaDB, unha restrición `UNIQUE` pode admitir varios valores `NULL`, xa que estes non se consideran valores iguais para esta comprobación.

2.4. NOT NULL

A restrición `NOT NULL` obriga a que unha columna teña sempre un valor. Non permite almacenar `NULL`.

```
CREATE TABLE empregados (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    apelidos VARCHAR(150) NOT NULL,
    telefono VARCHAR(20),
    data_alta DATE NOT NULL
) ENGINE=InnoDB;
```

Neste exemplo, o nome, os apelidos e a data de alta son obrigatorios. O teléfono é opcional.

Cómpre distinguir entre `NULL` e unha cadea baleira. Unha cadea baleira (`''`) é un valor de texto, mentres que `NULL` representa a ausencia dun valor.

```
-- Non permitido: nome obrigatorio.
INSERT INTO empregados (nome, apelidos, data_alta)
VALUES (NULL, 'Pérez García', '2026-01-10');

-- Permitido: teléfono opcional.
INSERT INTO empregados (nome, apelidos, telefono, data_alta)
VALUES ('Ana', 'Pérez García', NULL, '2026-01-10');
```

2.5. CHECK

A restrición `CHECK` permite definir condicións que deben cumprir os valores dunha columna ou dunha fila.

Utilízase para validar dominios de datos e regras sinxelas do negocio directamente na base de datos.

```
CREATE TABLE produtos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    prezo DECIMAL(10,2) NOT NULL,
    stock INT NOT NULL,
    CHECK (prezo >= 0),
    CHECK (stock >= 0)
) ENGINE=InnoDB;
```

Neste caso, non se permite introducir produtos cun prezo ou stock negativo.

Exemplo doutra condición:

```
CREATE TABLE alumnos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    edad INT NOT NULL,
    CHECK (edad >= 16 AND edad <= 100)
) ENGINE=InnoDB;
```

A idade debe estar comprendida entre 16 e 100 anos.

Tamén se poden definir restricións `CHECK` sobre varias columnas:

```
CREATE TABLE pedidos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    subtotal DECIMAL(10,2) NOT NULL,
    descuento DECIMAL(10,2) NOT NULL DEFAULT 0,
    CHECK (subtotal >= 0),
    CHECK (descuento >= 0),
    CHECK (descuento <= subtotal)
) ENGINE=InnoDB;
```

A restrición garante que o desconto non sexa negativo nin superior ao subtotal.

> **Nota:** nas versións modernas de MariaDB, as restricións `CHECK` son aplicadas polo servidor. Ademais, unha condición `CHECK` non substitúe a restrición `NOT NULL` : se unha expresión devolve `UNKNOWN` por conter `NULL` , a comprobación non a rexeita. Por iso, os campos obrigatorios deben declararse tamén como `NOT NULL` .

2.6. Comparación das restricións

Restrición	Finalidade	Exemplo
PRIMARY KEY	Identificar un rexistro de forma única.	Identificador dun alumno.
FOREIGN KEY	Garantir relacións entre táboas.	Curso ao que pertence un alumno.
UNIQUE	Evitar valores duplicados.	Correo electrónico.
NOT NULL	Obrigar a introducir un valor.	Nome dun alumno.
CHECK	Validar condicións sobre os datos.	Prezo maior ou igual a cero.

2.7. Engadir e eliminar restricións

As restricións poden definirse ao crear unha táboa ou engadirse posteriormente mediante `ALTER TABLE`.

Engadir unha restrición CHECK:

```
ALTER TABLE produtos
ADD CONSTRAINT chk_stock
CHECK (stock >= 0);
```

Engadir unha restrición UNIQUE:

```
ALTER TABLE usuarios
ADD CONSTRAINT uq_email
UNIQUE (email);
```

Engadir unha clave foránea:

```
ALTER TABLE alumnos
ADD CONSTRAINT fk_alumnos_cursos
FOREIGN KEY (curso_id)
REFERENCES cursos(id);
```

Eliminar unha clave foránea:

```
ALTER TABLE alumnos
DROP FOREIGN KEY fk_alumnos_cursos;
```

Eliminar unha restrición CHECK:

```
ALTER TABLE produtos
DROP CONSTRAINT chk_stock;
```

Para engadir unha restrición sobre unha táboa que xa contén datos, estes deben cumprir previamente a condición. Se existen rexistros incorrectos ou duplicados, MariaDB pode rexeitar a operación.

3. Medidas para garantir a consistencia das operacións

3.1. Que é a consistencia?

A **consistencia** é a propiedade pola que unha operación ou transacción mantén a base de datos nun estado válido, respectando todas as restricións e regras definidas.

Non abonda con que cada sentenza SQL se execute correctamente. Cando varias operacións forman parte dun mesmo proceso, todas deben completarse de maneira coherente.

> **Exemplo: rexistro dun pedido**
>
> Ao rexistrar un pedido nunha tenda, normalmente é necesario:
>
> 1. Crear o rexistro principal do pedido.
> 2. Inserir as liñas correspondentes aos produtos.
> 3. Comprobar que os produtos existen.
> 4. Verificar que hai stock suficiente.
> 5. Actualizar o stock dos produtos.
> 6. Calcular e gardar o importe total.
>
> Se se crea o pedido pero falla a inserción das liñas, a base de datos podería quedar cun pedido incompleto. Para evitar esta situación, todas as operacións deben formar parte dunha mesma transacción.

3.2. Validación dos datos

A validación consiste en comprobar que os valores introducidos cumpren as condicións necesarias antes de incorporalos ou utilizalos nunha operación.

As validacións poden realizarse en diferentes niveis:

Nivel	Responsabilidade	Exemplo
Aplicación	Comprobar os datos introducidos polo usuario.	Verificar que un formulario contén un correo válido.
Guión SQL	Comprobar condicións antes de executar operacións.	Verificar que existe stock suficiente.
Base de datos	Aplicar restricións de integridade.	CHECK, NOT NULL, UNIQUE e FOREIGN KEY.

É recomendable combinar estes niveis. A aplicación pode ofrecer mensaxes máis claras ao usuario, pero a base de datos debe manter as restricións fundamentais para impedir que calquera conexión introduza información incorrecta.

3.3. Uso de transaccións

As transaccións permiten agrupar varias operacións nunha única unidade de traballo. Ou ben se confirman todas, ou ben se desfán os cambios realizados.

As instrucións fundamentais son:

```
START TRANSACTION;

-- Operacións de modificación
INSERT INTO pedidos (...);
INSERT INTO detalle_pedido (...);
UPDATE produtos SET stock = ...;

-- Se todo é correcto:
COMMIT;

-- Se se produce algún problema:
ROLLBACK;
```

O uso de transaccións é especialmente importante cando se modifican varias táboas relacionadas.

3.4. Integridade referencial

As claves foráneas evitan que se creen relacións con rexistros inexistentes e permiten controlar o comportamento das eliminacións e modificacións.

Por exemplo, unha liña de pedido non debería referenciar un produto que non exista. A restrición de clave foránea impide que se garde unha relación incorrecta.

```
CREATE TABLE detalle_pedido (
    id INT AUTO_INCREMENT PRIMARY KEY,
    pedido_id INT NOT NULL,
    produto_id INT NOT NULL,
    cantidade INT NOT NULL,
    FOREIGN KEY (pedido_id)
        REFERENCES pedidos(id),
    FOREIGN KEY (produto_id)
        REFERENCES produtos(id),
    CHECK (cantidade > 0)
) ENGINE=InnoDB;
```

3.5. Control de erros

Durante a execución dun proceso poden producirse erros por diferentes motivos:

- Un rexistro que non existe.
- Unha clave duplicada.
- Unha violación dunha clave foránea.
- Un valor que incumpre unha restrición CHECK.
- Unha actualización que non afecta a ningún rexistro.
- Un problema de bloqueo ou un deadlock.
- Un erro de sintaxe ou de permisos.

Un proceso correcto debe detectar os erros e evitar que se confirmen cambios parciais.

3.6. Control de erros en procedementos almacenados

MariaDB permite utilizar xestores de condicións mediante `DECLARE ... HANDLER` para controlar erros durante a execución dun procedemento almacenado.

Un patrón habitual consiste en capturar os erros SQL, executar `ROLLBACK` e propagar o erro á aplicación.

```
DELIMITER //

CREATE PROCEDURE crear_operacion()
BEGIN

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    INSERT INTO pedidos (fecha)
    VALUES (CURRENT_DATE());

    INSERT INTO detalle_pedido
        (pedido_id, produto_id, cantidade)
    VALUES
        (LAST_INSERT_ID(), 1, 2);

    COMMIT;

END //

DELIMITER ;
```

Neste exemplo:

- `DECLARE EXIT HANDLER` define o tratamento dos erros SQL.
- `ROLLBACK` desfai os cambios da transacción.
- `RESIGNAL` volve lanzar o erro para que a aplicación poida detectalo.
- `COMMIT` só se executa se as operacións rematan correctamente.

> **Importante:** o exemplo é un patrón básico de tratamento de erros. Nun proceso real tamén se deben validar os identificadores, as cantidades, o stock e o resultado de cada operación antes de confirmar a transacción.

3.7. Comprobar o número de filas afectadas

Unha operación SQL pode executarse sen erros e, con todo, non modificar ningún rexistro porque a condición `WHERE` non atopou coincidencias.

Para comprobar o número de filas afectadas pódese utilizar `ROW_COUNT()`.

```
UPDATE produtos
SET stock = stock - 2
WHERE id = 1
  AND stock >= 2;

SELECT ROW_COUNT();
```

Se o resultado é 1, a operación afectou un rexistro. Se é 0, non se modificou ningunha fila.

Esta comprobación é importante en operacións como:

- Descontar stock dun produto.
- Actualizar o saldo dunha conta.
- Modificar o estado dun pedido.
- Asignar un alumno a un curso.

> **Precaución:** `ROW_COUNT()` debe consultarse inmediatamente despois da sentenza que se quere comprobar, xa que informa do resultado da última operación de modificación.

3.8. Consistencia e propiedades ACID

As propiedades ACID estudadas anteriormente tamén contribúen a garantir a consistencia:

- **Atomicidade:** todas as operacións dunha transacción se confirman ou se desfán.
- **Consistencia:** a transacción debe respectar as regras de integridade da base de datos.
- **Illamento:** as transaccións concorrentes non deben interferir de maneira incorrecta.
- **Durabilidade:** os cambios confirmados deben conservarse.

4. Práctica integradora: xestión de pedidos dunha tenda

4.1. Obxectivo

Nesta práctica desenvolverase un proceso completo de manipulación de datos para unha pequena tenda. O proceso permitirá rexistrar pedidos, consultar información de varias táboas, actualizar existencias e garantir que todas as operacións se realicen correctamente.

Utilizaranse os seguintes elementos:

- Creación e modificación de táboas.

- Restricións de integridade.
- Inserción, actualización e eliminación de rexistros.
- Subconsultas e combinacións de táboas.
- Procedementos almacenados e variables.
- Transaccións e control de erros.
- Consultas de comprobación dos resultados.

4.2. Deseño da base de datos

A base de datos debe representar unha tenda na que existen clientes, produtos, pedidos e liñas de pedido.

O modelo debe conter as seguintes táboas:

Táboa	Información almacenada	Relacións
clientes	Identificador, nome, correo electrónico e localidade.	Un cliente pode realizar varios pedidos.
produtos	Identificador, nome, prezo e stock.	Un produto pode aparecer en varias liñas de pedido.
pedidos	Identificador, cliente, data e estado.	Cada pedido pertence a un cliente.
detalle_pedido	Pedido, produto, cantidade e prezo aplicado.	Relaciona pedidos e produtos.

As relacións principais son:

- Clientes 1:N Pedidos.
- Pedidos 1:N Detalle_pedido.
- Produtos 1:N Detalle_pedido.
- Pedidos e produtos manteñen unha relación N:M a través de detalle_pedido.

4.3. Creación das táboas

O seguinte código mostra unha posible implementación do modelo. Todas as táboas utilizan InnoDB.

```
CREATE DATABASE IF NOT EXISTS tenda;
USE tenda;

CREATE TABLE clientes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    localidade VARCHAR(100)
) ENGINE=InnoDB;

CREATE TABLE produtos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    prezo DECIMAL(10,2) NOT NULL,
    stock INT NOT NULL DEFAULT 0,
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    CHECK (prezo >= 0),
    CHECK (stock >= 0)
) ENGINE=InnoDB;

CREATE TABLE pedidos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    cliente_id INT NOT NULL,
    data_pedido DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    estado VARCHAR(20) NOT NULL DEFAULT 'PENDENTE',
    total DECIMAL(10,2) NOT NULL DEFAULT 0,
    FOREIGN KEY (cliente_id)
        REFERENCES clientes(id),
    CHECK (total >= 0),
    CHECK (
        estado IN (
            'PENDENTE',
            'CONFIRMADO',
            'ENVIADO',
            'CANCELADO'
        )
    )
) ENGINE=InnoDB;

CREATE TABLE detalle_pedido (
    id INT AUTO_INCREMENT PRIMARY KEY,
    pedido_id INT NOT NULL,
    produto_id INT NOT NULL,
    cantidade INT NOT NULL,
    prezo_unitario DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (pedido_id)
        REFERENCES pedidos(id)
        ON DELETE CASCADE,
    FOREIGN KEY (produto_id)
        REFERENCES produtos(id),
    CHECK (cantidade > 0),
```

```
    CHECK (prezo_unitario >= 0)
) ENGINE=InnoDB;
```

O deseño incorpora:

- Claves primarias para identificar os rexistros.
- Claves foráneas para garantir as relacións.
- Restricións UNIQUE para evitar correos duplicados.
- Restricións NOT NULL para os datos obrigatorios.
- Restricións CHECK para validar prezos, cantidades, stock e estados.
- InnoDB para permitir transaccións e bloqueos.

4.4. Inserción de datos iniciais

Insire varios clientes e produtos para poder probar as operacións.

```
INSERT INTO clientes (nome, email, localidade)
VALUES
('Ana López', 'ana@exemplo.com', 'A Coruña'),
('Brais Pérez', 'brais@exemplo.com', 'Lugo'),
('Carla García', 'carla@exemplo.com', 'Vigo'),
('Diego Fernández', 'diego@exemplo.com', 'Ourense');

INSERT INTO produtos (nome, prezo, stock)
VALUES
('Teclado', 25.50, 20),
('Rato', 15.00, 30),
('Monitor', 180.00, 10),
('Disco SSD', 75.00, 15),
('Memoria USB', 8.50, 50);
```

4.5. Consultas e subconsultas

Antes de realizar modificacións, é necesario consultar os datos e identificar as relacións existentes.

Consultar os pedidos cos datos dos clientes

```
SELECT
    p.id AS pedido,
    c.nome AS cliente,
    p.data_pedido,
    p.estado,
    p.total
FROM pedidos p
INNER JOIN clientes c
    ON p.cliente_id = c.id;
```

Consultar os produtos cun stock inferior á media

```
SELECT nome, stock
FROM produtos
WHERE stock < (
    SELECT AVG(stock)
    FROM produtos
);
```

Consultar os clientes que realizaron pedidos

```
SELECT *
FROM clientes c
WHERE EXISTS (
    SELECT 1
    FROM pedidos p
    WHERE p.cliente_id = c.id
);
```

Consultar os produtos que nunca se venderon

```
SELECT *
FROM produtos pr
WHERE NOT EXISTS (
    SELECT 1
    FROM detalle_pedido dp
    WHERE dp.produto_id = pr.id
);
```

Consultar o importe total vendido por produto

```
SELECT
    pr.id,
    pr.nome,
    SUM(dp.cantidade) AS unidades_vendidas,
    SUM(dp.cantidade * dp.prezo_unitario) AS importe_total
FROM produtos pr
INNER JOIN detalle_pedido dp
    ON pr.id = dp.produto_id
GROUP BY pr.id, pr.nome
ORDER BY importe_total DESC;
```

4.6. Inserción de pedidos e liñas

Un pedido está formado por un rexistro principal e unha ou varias liñas. É necesario garantir que todas as liñas pertencen a un pedido existente.

Exemplo de creación dun pedido:

```
INSERT INTO pedidos (cliente_id)
VALUES (1);
```

O cliente co identificador 1 debe existir previamente. Despois de inserir o pedido, pódese obter o identificador xerado mediante `LAST_INSERT_ID()`.

```
SET @pedido_id = LAST_INSERT_ID();

INSERT INTO detalle_pedido
    (pedido_id, produto_id, cantidade, prezo_unitario)
VALUES
    (@pedido_id, 1, 2, 25.50),
    (@pedido_id, 2, 1, 15.00);
```

Neste caso, o pedido contén dous teclados e un rato. O prezo unitario queda gardado no detalle para conservar o prezo aplicado no momento da compra, aínda que o prezo actual do produto cambie posteriormente.

4.7. Actualización do total dun pedido

O importe total pode calcularse a partir das liñas de pedido. É posible actualizar o rexistro principal utilizando unha subconsulta:

```
UPDATE pedidos
SET total = (
    SELECT SUM(cantidade * prezo_unitario)
    FROM detalle_pedido
    WHERE pedido_id = 1
)
WHERE id = 1;
```

Tamén se pode utilizar unha combinación de táboas para actualizar o stock dos produtos en función das cantidades incluídas nun pedido.

```
UPDATE produtos p
INNER JOIN (
    SELECT
        produto_id,
        SUM(cantidade) AS unidades
    FROM detalle_pedido
    WHERE pedido_id = 1
    GROUP BY produto_id
) d ON p.id = d.produto_id
SET p.stock = p.stock - d.unidades;
```

Esta operación debe executarse só despois de comprobar que todos os produtos teñen stock suficiente. A comprobación e a actualización deben formar parte dunha mesma transacción.

4.8. Procedemento almacenado para rexistrar un pedido

O seguinte exemplo mostra como organizar varias operacións nun procedemento almacenado. O procedemento recibe un cliente, un produto e unha cantidade; comproba os datos, crea o pedido, insire o detalle, actualiza o stock e confirma a transacción.

```
DELIMITER //

CREATE PROCEDURE rexistrar_pedido(
    IN p_cliente_id INT,
    IN p_produto_id INT,
    IN p_cantidade INT
)
BEGIN

    DECLARE v_pedido_id INT;
    DECLARE v_prezo DECIMAL(10,2);
    DECLARE v_stock INT;
    DECLARE v_existe INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    -- Validar a cantidade.
    IF p_cantidade <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'A cantidade debe ser maior que cero';
    END IF;

    -- Comprobar que existe o cliente.
    SELECT COUNT(*)
    INTO v_existe
    FROM clientes
    WHERE id = p_cliente_id;

    IF v_existe = 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'O cliente non existe';
    END IF;

    -- Bloquear o produto e consultar o seu stock.
    SELECT prezo, stock
    INTO v_prezo, v_stock
    FROM produtos
    WHERE id = p_produto_id
      AND activo = TRUE
    FOR UPDATE;

    -- Comprobar o stock dispoñible.
    IF v_stock < p_cantidade THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Non hai stock suficiente';
```

```
    END IF;

    -- Crear o pedido.
    INSERT INTO pedidos (cliente_id)
    VALUES (p_cliente_id);

    SET v_pedido_id = LAST_INSERT_ID();

    -- Crear a liña do pedido.
    INSERT INTO detalle_pedido (
        pedido_id,
        produto_id,
        cantidade,
        prezo_unitario
    )
    VALUES (
        v_pedido_id,
        p_produto_id,
        p_cantidade,
        v_prezo
    );

    -- Descontar o stock.
    UPDATE produtos
    SET stock = stock - p_cantidade
    WHERE id = p_produto_id
      AND stock >= p_cantidade;

    IF ROW_COUNT() <> 1 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Non foi posible actualizar o stock';
    END IF;

    -- Actualizar o total.
    UPDATE pedidos
    SET total = p_cantidade * v_prezo,
        estado = 'CONFIRMADO'
    WHERE id = v_pedido_id;

    COMMIT;

END //

DELIMITER ;
```

O procedemento incorpora diferentes mecanismos de seguridade:

- Parámetros de entrada para identificar o cliente, o produto e a cantidade.
- Variables locais para almacenar o prezo, o stock e o identificador do pedido.

- Condicionais para validar a cantidade e a existencia do cliente.
- `SELECT ... FOR UPDATE` para bloquear o produto durante a operación.
- Unha transacción para agrupar todas as modificacións.
- Un xestor de erros que executa `ROLLBACK`.
- `SIGNAL` para xerar erros cando non se cumpren as condicións.
- `ROW_COUNT()` para comprobar que o stock foi actualizado.

O bloqueo do produto evita que dúas transaccións comproben simultaneamente o mesmo stock e realicen vendas incompatibles. A operación de actualización volve comprobar a cantidade dispoñible como medida adicional de seguridade.

Para rexistrar un pedido:

```
CALL rexistrar_pedido(1, 1, 2);
```

Este exemplo crea un pedido para o cliente 1 con dúas unidades do produto 1, sempre que o cliente e o produto existan e haxa stock suficiente.

> Este procedemento simplifica o proceso e permite engadir unha única liña de produto por pedido. Nun sistema real sería necesario ampliar a súa lóxica para admitir varias liñas, validar todos os produtos antes de confirmar e xestionar outros aspectos como descontos, impostos e cancelacións.

4.9. Comprobación dos resultados

Unha vez executado o procedemento, débense consultar as diferentes táboas para verificar que os cambios se realizaron correctamente.

Consultar os pedidos:

```
SELECT *
FROM pedidos
ORDER BY id DESC;
```

Consultar o detalle dos pedidos:

```
SELECT
    p.id AS pedido,
    c.nome AS cliente,
    pr.nome AS produto,
    dp.cantidade,
    dp.prezo_unitario,
    dp.cantidade * dp.prezo_unitario AS importe
FROM pedidos p
INNER JOIN clientes c
    ON p.cliente_id = c.id
INNER JOIN detalle_pedido dp
    ON p.id = dp.pedido_id
INNER JOIN produtos pr
    ON dp.produto_id = pr.id
ORDER BY p.id DESC;
```

Consultar o stock actualizado:

```
SELECT id, nome, stock
FROM produtos
ORDER BY id;
```

Comprobar os totais dos pedidos:

```
SELECT
    p.id,
    p.total,
    COALESCE(
        SUM(dp.cantidade * dp.prezo_unitario),
        0
    ) AS total_calculado
FROM pedidos p
LEFT JOIN detalle_pedido dp
    ON p.id = dp.pedido_id
GROUP BY p.id, p.total;
```

O total almacenado no pedido debe coincidir co total calculado a partir das súas liñas.

5. Boas prácticas para garantir a integridade e a consistencia

Ao deseñar procesos de manipulación de datos é recomendable aplicar as seguintes medidas:

1. **Definir correctamente o modelo relacional.** Establecer claves primarias, foráneas e relacións coherentes.
2. **Aplicar restricións na base de datos.** Utilizar PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL e CHECK.
3. **Validar os datos antes de modificalos.** Comprobar identificadores, cantidades, importes e estados.
4. **Utilizar transaccións.** Agrupar as operacións que forman parte dun mesmo proceso.
5. **Controlar os erros.** Executar ROLLBACK cando non se poida completar correctamente unha operación.
6. **Controlar a concorrencia.** Utilizar bloqueos cando varias conexións poidan modificar os mesmos datos.
7. **Comprobar as filas afectadas.** Verificar que as actualizacións e eliminacións afectan aos rexistros esperados.
8. **Verificar os resultados.** Consultar os datos finais e comprobar que se manteñen as regras do negocio.

6. Resumo final da UD4

Bloque	Contidos principais	Sentenzas e elementos
Manipulación de datos	Inserción, modificación e eliminación de rexistros.	INSERT, UPDATE, DELETE, INSERT ... SELECT.
Subconsultas e varias táboas	Modificación de datos utilizando consultas e relacións.	IN, EXISTS, subconsultas, JOIN.
Guións SQL	Variables, condicións, bucles e procedementos.	DECLARE, SET, IF, WHILE, CREATE PROCEDURE, CALL.
Transaccións	Operacións atómicas e recuperación de cambios.	START TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT.
Concorrencia	Bloqueos, illamento e control de accesos simultáneos.	FOR UPDATE, LOCK IN SHARE MODE, niveis de illamento.
Integridade	Validación e consistencia da información.	PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK.

A manipulación segura dos datos require combinar correctamente as operacións DML, as restricións de integridade, os procedementos almacenados, as transaccións e os mecanismos de concorrencia. O obxectivo final é garantir que a base de datos conserve sempre información válida e coherente, mesmo

cando se producen erros ou varias conexións traballan simultaneamente.